Coding Questions

Question 1:

Online Examination System Design and Implementation

Task

Design and implement an online examination system that allows a student to start an examination, answer multiple-choice questions, and submit the examination. Each examination consists of several questions, and a student can have only one submitted attempt for a particular examination. Once an attempt is submitted, its answers cannot be changed. The system must calculate the result after submission. The design should accommodate different question types (e.g., Multiple Choice, True/False) in the future without altering the core examination submission and evaluation workflow. Identify the classes, attributes, operations, relationships, and multiplicities. Implement the submission and evaluation behavior in a programming language of your choice. Explain the object interactions during the process.

Sample Input:

1. Student starts Examination 'Math Quiz'.
2. Student answers Question 1 (e.g., 'A').
3. Student answers Question 2 (e.g., 'C').
4. Student submits 'Math Quiz' attempt.

Expected Output: Examination 'Math Quiz' started by Student. Question 1 answered with 'A'. Question 2 answered with 'C'. Examination 'Math Quiz' submitted successfully. Result for 'Math Quiz' attempt: 1/2 correct (assuming Question 1 correct answer is 'A', Question 2 correct answer is 'B').

Instructions

Consider an Examination class managing Question objects. Think about an Attempt class to record a student's submission. Use an interface or abstract class for Question to support different types and polymorphism for evaluation. Model the relationship between Student, Examination, and Attempt. Pay attention to the state changes of an Attempt.

Starter code

// Implement: Online Examination System Design and Implementation

Question 2:

Vehicle Rental System with Dynamic Pricing

Task

Develop a vehicle rental service that enables customers to rent available vehicles. A vehicle cannot be rented if there is an active rental for it. Different categories of vehicles (e.g., Standard, Luxury, SUV) have distinct rental charge rules. The total rental amount depends on the selected vehicle category and the rental duration. After a vehicle is returned, it must become available again. The system should be designed to easily incorporate new vehicle categories and their specific pricing rules without modifying the existing rental workflow. Identify the core classes, their attributes, operations, relationships, and multiplicities. Implement the rental, charge calculation, and return behaviors, ensuring the extensibility requirement is met.

Sample Input:

1. Customer rents 'Luxury Car A' for 3 days.
2. Customer rents 'Standard Car B' for 5 days.
3. Customer returns 'Luxury Car A'.

Expected Output: Luxury Car A rented for 3 days. Total charge: $300.00 (example value). Standard Car B rented for 5 days. Total charge: $250.00 (example value). Luxury Car A returned. Now available.

Instructions

Consider a Vehicle hierarchy with an abstract Vehicle class and concrete StandardCar, LuxuryCar, etc. Implement a Rental class to manage rental details. Use polymorphism for calculating rental charges based on vehicle type. Think about how to manage vehicle availability and prevent double-booking. Consider a RentalService that orchestrates the process.

Starter code

// Implement: Vehicle Rental System with Dynamic Pricing

Question 3:

Hotel Booking and Cancellation System

Task

Design and implement a hotel booking system where customers can reserve available rooms. A single room cannot have multiple overlapping active reservations. Different room categories (e.g., Standard, Deluxe, Suite) utilize different pricing rules. The booking amount should be calculated based on the selected room category and the duration of the reservation. A reservation can only be cancelled before a specified cancellation deadline. Identify the necessary classes, their attributes, operations, relationships, and multiplicities. Implement the availability checking, booking, price calculation, and cancellation behaviors. Focus on handling overlapping reservations and enforcing cancellation rules.

Sample Input:

1. Customer requests to book 'Deluxe Room 101' from 2024-12-01 to 2024-12-05.
2. Customer requests to book 'Standard Room 205' from 2024-12-03 to 2024-12-07.
3. Customer attempts to book 'Deluxe Room 101' from 2024-12-03 to 2024-12-07 (overlapping).
4. Customer cancels reservation for 'Deluxe Room 101' (before cancellation point).

Expected Output: Deluxe Room 101 booked from 2024-12-01 to 2024-12-05. Total price: $800.00 (example value). Standard Room 205 booked from 2024-12-03 to 2024-12-07. Total price: $600.00 (example value). Booking failed: Deluxe Room 101 is not available for 2024-12-03 to 2024-12-07. Reservation for Deluxe Room 101 cancelled successfully.

Instructions

Consider Room, Reservation, and Customer classes. Room should have a category and a way to check availability for a given date range. Reservation should store details like dates, Room, Customer, and a status. Use a mechanism to check for date overlaps for room availability. Implement a Hotel or BookingManager class to manage rooms and reservations. Define different pricing strategies for room categories.

Starter code

// Implement: Hotel Booking and Cancellation System

Question 4:

Employee Leave Request Management

Task

An organization requires a system for employees to submit leave requests. Each leave request must include the requested dates and its current status (e.g., Pending, Approved, Rejected). All requests must undergo a review process before being approved or rejected. Once a request is approved or rejected, it cannot revert to the pending state. Different types of employees (e.g., Full-time, Part-time, Contract) may adhere to different leave policies, and the system should support adding new employee types without modifying the core leave processing workflow. Design and implement the submission and review workflow. Identify the classes, attributes, operations, relationships, multiplicities, and applicable inheritance or interface relationships. Ensure the system enforces state transitions and allows for future extensibility regarding employee types.

Sample Input:

1. Employee 'John Doe' (Full-time) submits leave request for 2024-10-10 to 2024-10-12 (Pending).
2. Manager reviews John Doe's request and approves it.
3. Employee 'Jane Smith' (Part-time) submits leave request for 2024-11-01 to 2024-11-05 (Pending).
4. Manager attempts to change John Doe's approved request back to pending.

Expected Output: Leave request submitted by John Doe for 2024-10-10 to 2024-10-12. Status: Pending. Leave request for John Doe approved. Status: Approved. Leave request submitted by Jane Smith for 2024-11-01 to 2024-11-05. Status: Pending. Cannot change status: Approved request cannot revert to Pending.

Instructions

Model Employee with different types, possibly using an interface or abstract class for common behavior and specialized classes for specific leave rules. A LeaveRequest class should manage dates and its status, potentially using a state pattern or clear state transitions. The LeaveManager or ApprovalService would handle the review process. Pay attention to the rules for status changes (e.g., 'once approved/rejected, cannot return to pending').

Starter code

// Implement: Employee Leave Request Management

Question 5:

Food Order and Flexible Payment System

Task

Develop a food-delivery system where customers can select items from restaurants and place orders. An order must contain at least one selected item before it can be placed. Each selected item has a quantity, and a customer can place multiple orders. Customers can complete payments using various methods (e.g., Credit Card, Digital Wallet, Cash on Delivery). Payment must succeed for an order to be considered successfully placed. Important order events (e.g., 'Order Placed', 'Payment Successful') should be communicated to the customer. The system must support the introduction of new payment methods in the future without altering the existing order-processing workflow. Design and implement the order-placement use case. Identify classes, attributes, operations, relationships, multiplicities, and applicable

inheritance/interface relationships. Explain object interactions and implement the complete workflow, including flexible payment processing and event communication.

Sample Input:

1. Customer creates an order, adds 'Pizza (Qty 2)' and 'Soda (Qty 1)'.
2. Customer attempts to place the order with an empty cart (should fail).
3. Customer places the order, attempts payment via Credit Card (simulate success).
4. Customer creates another order, adds 'Burger (Qty 1)', attempts payment via Digital Wallet (simulate failure).

Expected Output: Order created. Added Pizza (Qty 2), Soda (Qty 1). Cannot place order: Order must contain at least one item. Order placed successfully. Payment via Credit Card successful. Order status: Paid. Notification: Order #123 placed and paid. Order placed. Payment via Digital Wallet failed. Order status: Pending Payment. Notification: Order #124 placed, awaiting payment.

Instructions

Consider Customer, Order, LineItem, Restaurant, FoodItem classes. For payments, use an IPaymentMethod interface with concrete CreditCardPayment, DigitalWalletPayment, etc. classes. The Order class will orchestrate adding items and handling payment. Implement an event notification mechanism. Focus on the 'composition over inheritance' principle for order items and the 'dependency on abstractions' for payment methods.

Starter code

// Implement: Food Order and Flexible Payment System

Quiz Questions:

Question 1

A system processes various types of financial transactions (e.g., deposits, withdrawals, transfers). Each transaction type requires specific validation and processing logic, but they all share a common interface for initiation and status tracking. The system is designed to allow new transaction types to be added without modifying the core transaction processing engine. Which design principle and OOP concept combination primarily enables this flexibility and extensibility?

A Encapsulation and Data Hiding
B Composition and Aggregation
C Abstraction and Runtime Polymorphism
D Static Members and Object Identity

Question 2

Consider a Vehicle class that can be either a Car or a Motorcycle. Both Car and Motorcycle have distinct methods for calculating fuel efficiency. A function print_vehicle_details accepts a Vehicle object and calls a calculate_fuel_efficiency method on it, without knowing the specific type of vehicle at compile time. Which of the following OOP concepts are directly demonstrated in this scenario?

A Method Overloading

B Runtime Polymorphism

C Inheritance

D Dynamic Dispatch

E Object Identity

Question 3

A Book object in a library system needs to track its availability. When a member borrows the book, its status changes from 'Available' to 'Borrowed'. When returned, it changes back to 'Available'. A 'Borrowed' book cannot be borrowed again until it's returned. Which UML behavioral model is best suited to represent these distinct states and the transitions between them, along with the conditions for valid state changes?

A Sequence Diagram

B Activity Diagram

C Use Case Diagram

D State Diagram

Question 4

A Customer can place multiple Orders, but an Order is always placed by exactly one Customer. An Order cannot exist without being associated with a Customer. If a Customer is removed from the system, all their Orders should also be removed. Which UML relationship types are most appropriate for modeling the strong lifecycle dependency between Customer and Order in this scenario?

A Association

B Aggregation

C Composition

D Dependency

E Generalization

Question 5

A FoodOrder object encapsulates its LineItems. Each LineItem represents a specific food item with a quantity. The LineItems are created and managed exclusively by the FoodOrder object and cease to exist if the FoodOrder is cancelled or completed. No LineItem can exist without being part of a FoodOrder. Which type of relationship best describes the connection between FoodOrder and LineItem?

A Association

B Aggregation

C Dependency

D Composition

Question 6

A PaymentProcessor module needs to handle various payment methods (e.g., Credit Card, PayPal, Bank Transfer). The PaymentProcessor should interact with these methods through a common IPaymentMethod interface. When a new payment method is introduced, the PaymentProcessor's code should not need modification. Which OOP principle is most directly applied here to achieve extensibility?

A Encapsulation

B Inheritance

C Abstraction

D Cohesion

Question 7

During an online examination, a student starts an attempt, answers questions, and finally submits. After submission, the answers cannot be changed, and the attempt is evaluated. The flow of these actions (start, answer, submit, evaluate) and the sequence of interactions between the Student, Examination, and Submission objects are crucial. Which UML diagram type is best suited to visualize this step-by-step interaction and message flow?

A Class Diagram

B Activity Diagram

C Sequence Diagram

D Component Diagram

Question 8

An Employee object has an annual_salary attribute. The business rule states that the annual_salary cannot be set to a negative value. A set_salary method is used to update this attribute, which includes a validation check to ensure the salary is non-negative before updating the internal state. Which of the following OOP concepts are effectively demonstrated by this design?

A Data Hiding

B Encapsulation

C Inheritance

D Maintaining Valid Object State

E Abstraction

Question 9

A Logger interface defines a log_message(message: str) method. Concrete implementations include FileLogger, DatabaseLogger, and ConsoleLogger. A client application needs to log messages but should not depend on which specific logger implementation is being used. This approach allows for swapping logger implementations without altering the client code. What is the UML relationship between the FileLogger class and the Logger interface?

A Generalization

B Association

C Aggregation

D Realization

Question 10

A Course can have a maximum of 50 Students. A Student can enroll in multiple Courses. What is the correct multiplicity representation for the relationship between Course and Student from the perspective of Course to Student and Student to Course?

A Course 1 --- 1..* Student, Student 1 --- 1..* Course

B Course 0..* --- 1 Student, Student 0..* --- 1 Course

C Course 1 --- 0..50 Student, Student 1 --- 0..* Course

D Course 1 --- * Student, Student 1 --- * Course

Concept Questions

Question 1:

In a ShoppingCart object, the total_price should always be the sum of the prices of all LineItems. If an external operation could directly modify total_price without updating LineItems, the object's state could become inconsistent. Explain how the concept of 'invariants' relates to maintaining a valid object state within a ShoppingCart. Discuss how proper encapsulation and controlled modification of state, typically through methods, enforce these invariants and prevent invalid states.

Question 2:

A SmartDevice can have optional capabilities like Camera, GPS, or Bluetooth. If every capability were modeled using inheritance (e.g., SmartDeviceWithCamera, SmartDeviceWithGPS), it could lead to an explosion of classes and a rigid hierarchy, making it hard to manage combinations of capabilities (e.g., a device with both camera and GPS). Explain why composition might be a more suitable approach than inheritance for modeling such optional capabilities. Use the SmartDevice scenario to illustrate how composition provides greater flexibility and extensibility when dealing with varied combinations of features.

Question 3:

During a customer checkout process in an e-commerce system, a CheckoutManager object initiates a PaymentGateway interaction, which in turn communicates with an external BankService. The BankService returns a result to the PaymentGateway, and finally, the PaymentGateway relays the payment status back to the CheckoutManager. Describe how UML Sequence Diagram reasoning would represent this flow of interactions. Specifically, explain how message

ordering, object lifelines, and return messages are depicted, and why this diagram is effective for understanding dynamic object collaboration in this scenario.

Question 4:

Consider a NotificationService that can send messages using various mechanisms such as email, SMS, or push notifications. A client application needs to send a notification but should not be aware of the specific underlying mechanism. Explain how the OOP concept of abstraction, specifically through interfaces or abstract classes, allows the NotificationService to be highly extensible. Discuss how this design makes it possible to introduce new notification mechanisms (e.g., a new messaging app integration) without requiring any changes to the client's code, demonstrating the benefits of dependency on abstractions.

Question 5:

A University contains Departments, and each Department has Professors. If a University ceases to exist, its Departments also cease to exist, but the Professors might continue their careers at other institutions. Explain the difference between Aggregation and Composition UML relationships using this University, Department, and Professor scenario. Clearly articulate the concepts of object ownership and lifecycle dependency that differentiate these two types of 'whole-part' relationships and justify which one applies where.